

Day 12 Fastapi

Rohit Raj Singh

What is FastAPI?

FastAPI is a modern Python web framework used to build APIs quickly, with automatic validation and documentation.

Why is it called **FastAPI**?

- **Fast** → FastAPI has high performance, especially for I/O-bound applications, and supporting asynchronous programming with `async/await`.
- **API** → It is specifically designed for building Web APIs.

Key Advantages

Feature	Meaning
High Performance	Handles requests efficiently and supports asynchronous programming with <code>async/await</code> .
Easy to Use	Simple Python syntax makes API development easier.
Automatic Validation	Pydantic validates request data automatically.
Automatic Docs	Generates Swagger UI (<code>/docs</code>) automatically.
Type Hints	Python type hints define and validate API parameters clearly.
Async Support	Supports <code>async/await</code> for I/O-heavy operations.
Dependency Injection	Easily manages reusable dependencies like database sessions and authentication.
OpenAPI Support	Automatically generates an OpenAPI specification for the API.
Production Ready	Suitable for building scalable backend services and APIs.

FastAPI is a high-performance Python framework for building APIs with automatic validation, documentation, and strong support for asynchronous programming.



FastAPI Project Structure

- **app/** – Main backend application package.
- **main.py** – Entry point that creates the FastAPI app and connects routers.

- **api/** – Contains API routing.
- **v1/router.py** – Central router that connects different API endpoints.
- **endpoints/** – Contains actual API route functions; `employees.py` handles Employee APIs.
- **core/** – Common configuration and dependencies.
- **config.py** – Loads application settings such as the database URL.
- **dependencies.py** – Provides reusable dependencies such as database sessions.
- **db/** – Database setup and connection management.
- **base.py** – Defines the SQLAlchemy base class for models.
- **session.py** – Creates the database engine and session.
- **models/** – Defines SQLAlchemy models representing database tables.
- **schemas/** – Defines Pydantic request and response models.
- **services/** – Contains business/application logic.
- **repositories/** – Handles database queries and CRUD operations.
- **tests/** – Contains automated API tests.
- **requirements.txt** – Lists required Python packages.
- **README.md** – Contains project documentation.
- **.env** – Stores environment configuration such as database credentials.
- **__init__.py** – Marks directories as Python packages.
- **__pycache__** – Stores Python's automatically generated compiled cache files.

Connections

main.py
↓
router.py
↓
endpoints/
↓
schemas/

↓
services/
↓
repositories/
↓
models/
↓
db/
↓
PostgreSQL

Meaning

main.py

→ Starts the application

router.py

→ Finds which API module handles the URL

endpoints/

→ Handles the HTTP request

schemas/

→ Validates request and response data

services/

→ Applies business logic

repositories/

→ Performs database operations

models/

→ Represents database tables

db/

→ Connects and manages database sessions

PostgreSQL

→ Actually stores the data

main → router → endpoint → schema → service → repository → model → database

API Routes and Types

API Route: A combination of an **HTTP method + URL path** that tells FastAPI which endpoint function should handle a request.

Routes map an HTTP method and URL to a specific endpoint function.

Common API Route Types

HTTP Method	Purpose	Example
GET	Retrieve data	GET /employees
POST	Create new data	POST /employees
PUT	Replace/update existing data	PUT /employees/1
PATCH	Partially update data	PATCH /employees/1
DELETE	Delete data	DELETE /employees/1

Dynamic Routes

A **dynamic route** is a route where part of the URL changes based on the value provided by the client, like an **ID**.

Example

```
@router.get("/employees/{employee_id}")
def get_employee(employee_id: int):
    return {"id": employee_id}
```

Path Parameters in FastAPI

A **path parameter** is a value that is placed **inside the URL path** and is used to identify a specific resource.

a **path parameter** is a variable embedded directly within the URL path to identify a specific resource. They allow you to capture dynamic values from the URL and pass them directly into your Python function as arguments.

For example:

GET /employees/101

Here, 101 is the **path parameter** — it identifies employee 101.

In FastAPI:

```
from fastapi import FastAPI
```

```
app = FastAPI()
```

```
@app.get("/employees/{employee_id}")
def get_employee(employee_id: int):
    return {"employee_id": employee_id}
```

Query Parameters in FastAPI

A **query parameter** is a parameter passed to an API through the URL **after the ? symbol**.

It is mainly used to **filter, search, sort, paginate, or customize the data returned by an API.**

Example:

GET /employees?department=IT

Here:

department=IT

is the **query parameter**.

```
@app.get("/employees")
def get_employees(department: str, limit: int = 10):
    return {"department": department, "limit": limit}
```

Common Uses

Filtering → /employees?department=IT

Searching → /employees?search=Rohit

Pagination → /employees?limit=10&offset=20

Sorting → /employees?sort=name

If a function argument is **not declared as a path parameter in the route path**, FastAPI generally interprets it as a **query parameter**.

- **Optional parameter:** The caller can **skip the parameter**, and its value can be None.

department: str = None

If not provided → department = None

- **Default parameter:** The caller can **skip the parameter**, and a predefined value is used.

limit: int = 10

If not provided → limit = 10

Simple difference

Optional → can be absent and becomes None; **Default** → can be absent but gets a specified value.

Request Models in FastAPI

A **Request Model** is a **Pydantic model** that defines the structure and data types of information the client is allowed to send to an **API**. FastAPI uses it to automatically **validate the incoming request data**.

```
class EmployeeCreate(BaseModel):  
    name: str  
    age: int
```

Example request:

```
{  
    "name": "Rohit",  
    "age": 21  
}
```

If the data doesn't match the defined structure or types, FastAPI returns a **validation error**.

In short:

Request Model = **Defines + validates incoming API data**.

Response Models in FastAPI

A **Response Model** is a **Pydantic model** that defines the structure and data types of the data an **API** should return to the client.

```
class EmployeeResponse(BaseModel):  
    id: int  
    name: str
```

Use it in an endpoint:

```
@app.get("/employees/{id}", response_model=EmployeeResponse)  
def get_employee(id: int):  
    return {"id": id, "name": "Rohit"}
```

FastAPI uses the response model to **validate and format the returned data**.

In short:

Request Model → **validates incoming data**

Response Model → **validates and structures outgoing data**

Advantages of Response Models

- **Controls response structure** — Defines exactly what data the API returns.
- **Validates output** — Ensures returned data matches the expected types.
- **Improves security** — Prevents accidentally exposing unwanted fields.
- **Consistent responses** — Keeps API responses predictable for frontend clients.

Pydantic Validation

Pydantic validation checks whether the data received by an API matches the **defined structure, data types, and rules** in a Pydantic model.

Pydantic validation is the process of **checking incoming or outgoing data against the types and rules defined in a Pydantic model**.

Pydantic uses: Data Validation | Data Parsing | Request Models | Response Models | Type Checking | Field Constraints | Automatic Error Handling | predefined structure

Small FastAPI Example

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Employee(BaseModel):
    name: str
    age: int

@app.post("/employees")
def create_employee(employee: Employee):
    return employee
```

Benefits of Pydantic

- **Data Validation** — Checks whether data matches the expected types and rules.
- **Type Safety** — Uses Python type hints such as str, int, float, etc.
- **Structured Data** — Defines a clear structure for API requests and responses.
- **Automatic Errors** — Invalid data produces clear validation errors.
- **Serialization** — Converts validated Python objects into JSON-compatible data.
- **Swagger Integration** — Models automatically appear in FastAPI's /docs.

Swagger / OpenAPI

OpenAPI is a standard used to **describe and document an API**, including its endpoints, parameters, request/response formats, and status codes.

Swagger UI is a **web interface that displays the OpenAPI documentation and lets you test the API directly from the browser.**

In FastAPI

When we run:

```
uvicorn app.main:app --reload
```

Open:

<http://127.0.0.1:8000/docs>

we will see your API endpoints, for example:

POST /employees

GET /employees

GET /employees/{id}

PUT /employees/{id}

DELETE /employees/{id}

Key difference

OpenAPI = specification/standard that describes the API.

Swagger UI = interactive interface that displays and tests that OpenAPI documentation.

In short: FastAPI automatically generates **OpenAPI documentation** and provides **Swagger UI** at /docs.

Dependency Concepts in FastAPI

Dependency Injection is a FastAPI mechanism where an endpoint declares a functionality it needs, and FastAPI **automatically executes that dependency and provides its result to the endpoint.**

Dependency Injection in FastAPI means that we **create a reusable function for a required functionality (such as creating a database session or checking authentication) once, and whenever an endpoint needs that functionality, we declare it using Depends() so FastAPI automatically executes the dependency, provides its result to the endpoint, and then the endpoint continues with its own business logic.**

When one function/endpoint needs another functionality to be completed first, we define that functionality as a dependency and use Depends() so FastAPI executes it first, provides its result to the endpoint, and then the endpoint continues its main work.

Dependencies are useful when the same functionality is required by multiple endpoints, such as **database sessions, authentication, permissions, or common request processing.**

Example

```
from fastapi import Depends
```

```
@app.get("/employees")
def get_employees(db :Session= Depends(get_db)):
    return {"message": "Employees fetched"}
```

How it works

Request



FastAPI sees Depends(get_db)



get_db() executes



Database session is provided



get_employees(db) executes



Response

Why use Dependencies?

- **Reuse** common functionality.
- **Avoid repeating** the same setup logic.
- **Keep endpoints clean** and focused on business logic.
- **Manage resources** such as database sessions properly.
- **Centralize authentication and authorization** logic.

API Documentation

Employee Management API

1. API Overview

The Employee Management API is a REST API developed using FastAPI to manage employee records through CRUD operations.

2. Base URL

<http://127.0.0.1:8000>

3. API Endpoints

Method	Endpoint	Description
POST	/employees	Create a new employee
GET	/employees	Retrieve all employees
GET	/employees/{emp_id}	Retrieve an employee by ID
PUT	/employees/{emp_id}	Update an employee
DELETE	/employees/{emp_id}	Delete an employee

4. Employee Data Structure

An employee record contains the following fields:

Field	Type	Description
id	Integer	Unique employee ID generated by the application

name	String	Employee name
email	String	Employee email address
department	String	Employee department
designation	String	Employee designation
salary	Float	Employee salary
is_active	Boolean	Employee active status

Example Employee

```
{  
  "id": 1,  
  "name": "Rohit Singh",  
  "email": "rohit@example.com",  
  "department": "Engineering",  
  "designation": "Software Engineer",  
  "salary": 50000,  
  "is_active": true  
}
```

5. POST /employees

Description

Creates a new employee.

The employee ID is automatically generated by the application.

Request Body

```
{  
  "name": "Rohit Singh",  
  "email": "rohit@example.com",
```

```
"department": "Engineering",  
"designation": "Software Engineer",  
"salary": 50000,  
"is_active": true  
}
```

Successful Response

Status Code: 201 Created

```
{  
  "id": 1,  
  "name": "Rohit Singh",  
  "email": "rohit@example.com",  
  "department": "Engineering",  
  "designation": "Software Engineer",  
  "salary": 50000,  
  "is_active": true  
}
```

Validation

The following validation rules are applied:

- name must contain between 2 and 50 characters.
- email must be a valid email address.
- department must contain between 2 and 50 characters.
- designation must contain between 2 and 50 characters.
- salary must be greater than 0.
- is_active must be a Boolean value.

Invalid request data results in a 422 Unprocessable Entity response.

6. GET /employees

Description

Retrieves all employees currently stored in the application.

Request

GET /employees

Successful Response

Status Code: 200 OK

```
[
  {
    "id": 1,
    "name": "Rohit Singh",
    "email": "rohit@example.com",
    "department": "Engineering",
    "designation": "Software Engineer",
    "salary": 50000,
    "is_active": true
  },
  {
    "id": 2,
    "name": "Priti Singh",
    "email": "priti.singh@example.com",
    "department": "HR",
    "designation": "HR Executive",
    "salary": 42000,
    "is_active": true
  }
]
```

If no employees exist, the API returns an empty list with status code 200 OK.

[]

7. GET /employees/{emp_id}

Description

Retrieves a specific employee using the employee ID.

Path Parameter

Parameter	Type	Description
emp_id	Integer	ID of the employee to retrieve

Example Request

GET /employees/1

Successful Response

Status Code: 200 OK

```
{
  "id": 1,
  "name": "Rohit Singh",
  "email": "rohit@example.com",
  "department": "Engineering",
  "designation": "Software Engineer",
  "salary": 50000,
  "is_active": true
}
```

Employee Not Found

Status Code: 404 Not Found

```
{  
  "detail": "Employee not found"  
}
```

Invalid Employee ID

If the ID provided in the URL is not an integer, FastAPI returns a validation error.

Status Code: 422 Unprocessable Entity

8. PUT /employees/{emp_id}

Description

Updates an existing employee using the employee ID.

Path Parameter

Parameter	Type	Description
emp_id	Integer	ID of the employee to update

Example Request

PUT /employees/1

Request Body

```
{  
  "name": "Rohit Singh",  
  "email": "rohit@example.com",  
}
```

```
"department": "Engineering",  
"designation": "Senior Software Engineer",  
"salary": 60000,  
"is_active": true  
}
```

Successful Response

Status Code: 200 OK

```
{  
  "id": 1,  
  "name": "Rohit Singh",  
  "email": "rohit@example.com",  
  "department": "Engineering",  
  "designation": "Senior Software Engineer",  
  "salary": 60000,  
  "is_active": true  
}
```

Employee Not Found

Status Code: 404 Not Found

```
{  
  "detail": "employee not found"  
}
```

Validation Error

Invalid request body data results in:

Status Code: 422 Unprocessable Entity

9. DELETE /employees/{emp_id}

Description

Deletes an existing employee using the employee ID.

Path Parameter

Parameter	Type	Description
emp_id	Integer	ID of the employee to delete

Example Request

DELETE /employees/1

Successful Response

Status Code: 204 No Content

No response body is returned.

Employee Not Found

Status Code: 404 Not Found

```
{  
  "detail": "employee not found"  
}
```

10. HTTP Status Codes

Status Code	Meaning	Usage
200 OK	Request successful	GET and PUT
201 Created	Resource created	POST

204 No Content	Resource deleted successfully	DELETE
404 Not Found	Employee does not exist	GET, PUT, DELETE
422 Unprocessable Entity	Validation failed	Invalid request data or path parameter

11. Request Models

EmployeeCreate

Used by:

POST /employees

Fields:

- name
- email
- department
- designation
- salary
- is_active

The id field is not included because the application generates it automatically.

EmployeeUpdate

Used by:

PUT /employees/{emp_id}

Fields:

- name
- email
- department

- designation
- salary
- is_active

EmployeeResponse

Used as the response model for the API.

Fields:

- id
- name
- email
- department
- designation
- salary
- is_active

12. Error Handling

The API uses HTTPException for employees that are not found.

Example:

```
raise HTTPException(  
    status_code=404,  
    detail="Employee not found"  
)
```

FastAPI and Pydantic automatically handle validation errors for invalid request data.

13. Data Storage

The application currently uses an in-memory Python list for storing employee records.

```
emp_db: list[dict] = []
```

Employee IDs are generated using:

```
seq_id = 0
```

A new employee receives the next available ID when created.

The data is temporary and is lost whenever the application is restarted.

14. Swagger Documentation

FastAPI automatically generates interactive API documentation.

Swagger UI is available at:

<http://127.0.0.1:8000/docs>

The following endpoints can be tested through Swagger:

```
POST  /employees
GET   /employees
GET   /employees/{emp_id}
PUT   /employees/{emp_id}
DELETE /employees/{emp_id}
```

OpenAPI specification is available at:

<http://127.0.0.1:8000/openapi.json>

15. API Testing

The following test cases can be performed through Swagger UI.

Create Employee

Valid data:

POST /employees → 201 Created

Invalid email:

POST /employees → 422 Unprocessable Entity

Negative salary:

POST /employees → 422 Unprocessable Entity

Get Employee

Existing employee:

GET /employees/1 → 200 OK

Non-existing employee:

GET /employees/999 → 404 Not Found

Update Employee

Existing employee:

PUT /employees/1 → 200 OK

Non-existing employee:

PUT /employees/999 → 404 Not Found

Delete Employee

Existing employee:

DELETE /employees/1 → 204 No Content

Non-existing employee:

DELETE /employees/999 → 404 Not Found

16. API Request-Response Flow

Client



HTTP Request



FastAPI Route



Path Parameter / Request Body



Pydantic Validation



CRUD Operation



Employee Data



Response Model



HTTP Response

